

MINIPROYECTOS 2026

Desarrollo asistido por Inteligencia Artificial de un resolvedor de sistemas de ecuaciones complejas para análisis de circuitos

Clave del miniproyecto: MP-26-070

Título del miniproyecto: Desarrollo asistido por Inteligencia Artificial de un resolvedor de sistemas de ecuaciones complejas para análisis de circuitos

Autores: Iker Emiliano Barrera García, Orlando Contreras Reyes

Adscripciones: Departamento de Sistemas Electrónicos, Centro de Ciencias Básicas, Universidad Autónoma de Aguascalientes

Introduccion

El análisis de circuitos es una de las tareas más importantes del ingeniero, pues aplica conceptos matemáticos y físicos para obtener parámetros como voltaje o corriente, siendo la base de gran parte de la tecnología actual. Sin embargo, resolver de forma eficiente los sistemas de ecuaciones complejas que surgen en el análisis de CA rara vez cuenta con herramientas prácticas y portátiles dentro del aula. Las soluciones habituales han sido comprar una calculadora especializada, usar simuladores de calculadora en el celular (como Droid48, que emula la HP-48G) o aplicar manualmente la regla de Cramer.

Estas soluciones presentan limitaciones claras: la regla de Cramer es poco eficiente; las calculadoras físicas y simuladas tienen un ingreso de datos tedioso y muestran resultados solo en forma rectangular o polar, nunca ambas y su uso indiscriminado puede derivar en deshonestidad académica cuando se recurre a la IA o a terceros sin comprender el proceso.

Ante esto, un equipo de estudiantes desarrolló una aplicación de escritorio, empaquetada como ejecutable (.exe), como primer prototipo hacia futuras versiones en Android y hardware dedicado. Cabe destacar que este prototipo fue diseñado en conjunto con IA, lo que demuestra el alcance de un ingeniero electrónico apoyado en estas herramientas.

Objetivos

Objetivo General

Desarrollar, con apoyo de herramientas de inteligencia artificial, una calculadora de escritorio capaz de resolver sistemas de ecuaciones con números complejos aplicados al análisis de circuitos de CA (mallas y nodos), mediante álgebra lineal numérica, con interfaz gráfica y distribución como ejecutable independiente para Windows y GNU/Linux.

Objetivos Específicos

- Implementar el ingreso de datos complejos en forma rectangular ($a + jb$) y polar ($r \angle \theta^\circ$).
- Resolver sistemas de $n \times n$ con coeficientes complejos mediante `numpy.linalg.solve`, evitando la regla de Cramer manual.
- Diseñar una interfaz con CustomTkinter que muestre resultados en ambas representaciones simultáneamente.
- Empaquetar la aplicación como ejecutable autónomo mediante PyInstaller.
- Validar la herramienta contra ejercicios de análisis de circuitos resueltos manualmente.

Materiales y metodos

El desarrollo se dividió en tres fases: selección de herramientas y arquitectura, implementación de los algoritmos con apoyo de IA, y construcción de la interfaz gráfica con su empaquetado.

Materiales y Herramientas

- **Entorno y lenguaje:** Visual Studio Code v1.85+, Python 3.10+.
- **Librerías:** NumPy (resolución de sistemas complejos), CustomTkinter/Tkinter (GUI), Pillow (recursos gráficos), OS/JSON (persistencia e historiales).
- **IA:** Asistente de código ChatGPT (OpenAI), usado para optimizar fragmentos, refactorizar y agilizar el diseño de la GUI.
- **Compilación:** PyInstaller, para generar el ejecutable autónomo.

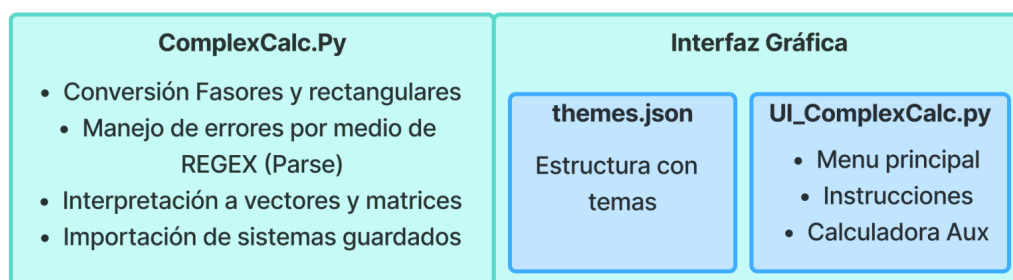


Figura 1: Arquitectura

Arquitectura y Algoritmos

Un sistema de mallas o nodos de $n \times n$ se representa matricialmente como:

$$[\mathbf{Z}][\mathbf{I}] = [\mathbf{V}] \quad \rightarrow \quad [\mathbf{I}] = [\mathbf{Z}]^{-1}[\mathbf{V}] \quad (1)$$

donde $[\mathbf{Z}]$ es la matriz de impedancias complejas, $[\mathbf{I}]$ el vector de corrientes desconocidas y $[\mathbf{V}]$ el vector de fuentes. La resolución se realiza con el solver de NumPy, garantizando precisión y eficiencia frente a la regla de Cramer. Una vez calculado el vector solución, cada componente se convierte automáticamente a forma polar ($r = \sqrt{a^2 + b^2}$, $\theta = \arctan(b/a)$) para mostrar ambos formatos al usuario.

Flujo de Trabajo con IA

El desarrollo integró un ciclo iterativo con ChatGPT: generación de maquetas iniciales (ventanas, contenedores, celdas matriciales editables), resolución de errores al manipular arreglos complejos en tiempo real, refactorización para separar cálculo y renderizado, y supervisión/validación humana de cada segmento generado, probado contra ejercicios teóricos.

Interfaz y Empaquetado

Se adoptó CustomTkinter con un archivo .json de estilos, agregando una barra de herramientas para guardar matrices, limpiar celdas y alternar el formato de salida. El programa se compiló con PyInstaller (`--onefile --noconsole`), empaquetando intérprete, dependencias y recursos en un solo ejecutable.

Filosofía

El proyecto se guió por las filosofías *Open Source* y *Vibe Coding*: con solo bases de C para microcontroladores, los autores construyeron una herramienta funcional apoyándose en IA, demostrando que unas buenas bases de programación bastan para desarrollar soluciones prácticas. Se optó por Python por agilizar la implementación sin sacrificar funcionalidad. **Aclaración**

de autoría y licenciamiento: Parte de las estructuras de código fuente e interfaz fueron desarrolladas con apoyo del modelo de inteligencia artificial ChatGPT.

Resultados y discusiones

La calculadora facilita la resolución de sistemas lineales reales y complejos, siendo útil en cursos como Análisis de Circuitos, Electrónica, Señales y Sistemas y Métodos Matemáticos, donde el manejo de cantidades fasoriales es fundamental. Cuenta con historial de operaciones y, al ser gratuita y multiplataforma, resulta una alternativa accesible en el ámbito académico.

En la Figura 2 se muestra la solución de un sistema y la interfaz de la calculadora.

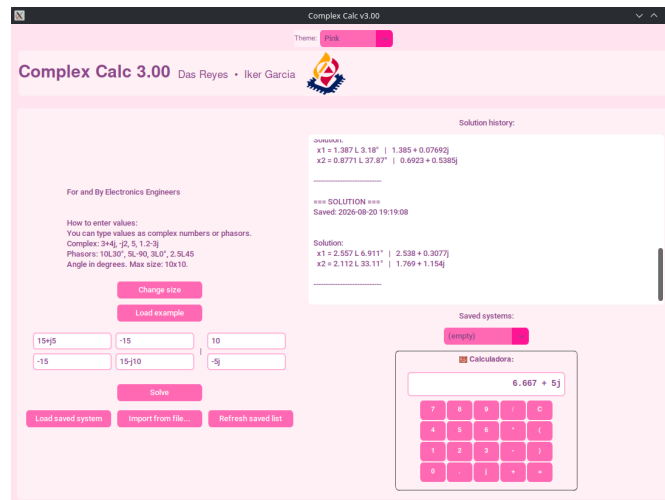


Figura 2: Solución de un sistema complejo.

Durante el desarrollo surgieron tres retos principales: lograr una interfaz intuitiva para usuarios sin experiencia, corregir errores en el guardado y carga de matrices, y personalizar visualmente Tkinter modificando el archivo `.json` de estilos.

Conclusiones

El proyecto comprobó que la inteligencia artificial es útil en programación cuando se usa como apoyo y no como sustituto del conocimiento del programador: partiendo solo de bases de C para microcontroladores, fue posible construir y validar una aplicación funcional para resolver sistemas de ecuaciones complejas aplicados al análisis de circuitos. Se obtuvo una herramienta que convierte números complejos entre forma rectangular y polar mediante NumPy, distribuida como ejecutable autónomo sin necesidad de configurar un entorno de programación.

Referencias

- [1] Anton, H., Rorres, C. (2013). *Álgebra lineal con aplicaciones*. Wiley.
- [2] Lay, D. C. (2012). *Álgebra lineal y sus aplicaciones*. 4.^a ed., Pearson.
- [3] NumPy Developers. (2026). *numpy.linalg.solve* — NumPy v2.5 Manual. <https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html>
- [4] Python Software Foundation. (2026). *tkinter* — Python interface to Tcl/Tk. <https://docs.python.org/3/library/tkinter.html>
- [5] Cortesi, D., Bajo, G., McMillan, W. (2026). *PyInstaller Manual*. <https://pyinstaller.org/en/stable/>
- [6] OpenAI. (2026). *ChatGPT*. <https://chatgpt.com/>